Project — Auditable AI Agents

Working title: Auditable AI Agents: An Immutable On-Chain Audit Trail for LLM Agent Actions

Author: Manikandan Sankaranarayanan
Program: Master of Engineering (Computer Science and Engineering)
Institution: S Veersamy Chettiya College of Engineering
Contact: smak07.s@gmail.com
Horizon: ~4 months
Emphasis: Provable tamper detection + measured cost and overhead (not a new blockchain)

---

1. What this project is

Autonomous LLM agents call tools, make decisions and act on behalf of users. Today their records live in ordinary log files or databases that whoever controls the server can edit or delete, and nobody can prove afterwards that a log was left untouched.

This project builds a **tamper-evident audit trail**:

1. A logger wraps an LLM agent and records every step (prompt, model response, tool call, tool result, timestamp).
2. Entries are **hash-chained**: each entry includes the hash of the previous one.
3. Batches of entry hashes are folded into a **Merkle root** and anchored in a smart contract.
4. A **verifier** recomputes hashes from the stored logs and compares them with the on-chain roots, reporting "intact" or the exact entry that was altered.

The question it answers:

Can we prove what an AI agent did, at what cost, and with how much overhead?

2. Core ideas (plain language)

- **Hash:** a short fixed-length fingerprint of data. Changing one character changes it completely.
- **Hash chain:** each entry carries the previous entry's hash, so removing or reordering one breaks everything after it.
- **Merkle tree:** many hashes folded into one root. A short proof can show a single entry belongs to the root without revealing the others.
- **Anchoring:** writing the root to a blockchain, where it cannot be changed unnoticed.
- **Off-chain logs:** the full data stays in ordinary storage. Only hashes go on-chain.

3. Deliverables

1. Python library: canonical serialisation, hash chain, Merkle tree and proofs.
2. Solidity contract that stores batch roots with timestamps.

3. Logger that wraps a demo LLM agent.
4. Verifier that checks logs against the chain and locates tampering.
5. Streamlit dashboard: timeline of actions with verified / tampered status.
6. Evaluation: detection tests, gas cost versus batch size, overhead, scaling.
7. Thesis and IEEE abstract.

4. Success criteria (what "done" means)

- All tamper scenarios (edit, delete, insert, reorder, truncate) are detected, and the first bad entry is located correctly, shown by automated tests.
- Python and contract hashing agree on shared test vectors.
- Gas, latency and scaling results are measured and reported, whatever they turn out to be.
- A live demo: edit a log file and watch the verifier flag it.

5. Honest limits

See `decisions.md` scope limits. Integrity is not the same as truth.

6. Plan

- **Month 1:** hash chain, Merkle tree, proofs, anchor contract draft, tests.
- **Month 2:** local chain setup, logger, demo agent, anchoring pipeline.
- **Month 3:** verifier, tamper tests, gas and overhead measurements.
- **Month 4:** dashboard, evaluation write-up, optional signed entries or selective disclosure.